

Reviewer Pack — Graph Energy Lower Bound on Tseitin Resolution Length

Entry e90418e8fa5a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 15:39:41 UTC

Graph Energy Lower Bound on Tseitin Resolution Length

Entry ID: e90418e8fa5a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 15:39:41 UTC

1. Conjecture

Field A (mathematical branch): SPECTRAL_GRAPH_THEORY **Field B** (complexity object): RESOLUTION_LENGTH

Statement:

For a graph G with adjacency matrix A , let $\text{Energy}(G) = \sum_{i=1}^n |\lambda_i|$ where λ_i are the eigenvalues of A . The Tseitin formula on G requires Resolution length $\geq 2^{\Omega(\text{Energy}(G))}$.

Rationale (proposer's reasoning):

Graph energy, a spectral invariant, captures structural complexity. High energy implies non-trivial eigenvalue distribution, which may correlate with the exponential growth of resolution refutations needed to handle the formula's dependencies.

Taxonomy category: TSEITIN_RES (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 25cb678ac90c6587

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -graph energy resolution length Tseitin formula -spectral graph theory graph energy resolution lower bound -exponential lower bound Tseitin formula resolution complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i + max(range(i, n), key=lambda x: abs(A[x][i]))
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

def determinant(A):
    n = len(A)
    det = 0
    if n == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += ((-1) ** i) * A[0][i] * determinant(submatrix)
    return det

def eigenvalues(A):
    n = len(A)
    if n == 2:
        return [A[0][0], A[1][1]]
    else:
        # Use QR algorithm for larger matrices
        Q, R = qr_decomposition(A)
        return eigenvalues(R)

def qr_decomposition(A):
    n = len(A)
    Q = [[0] * n for _ in range(n)]
    R = [row[:] for row in A]

    for k in range(n):
```

```

v = [R[i][k] for i in range(k, n)]
norm_v = math.sqrt(sum(x**2 for x in v))
v[k] += norm_v

Q[k][k] = v[k] / norm_v
for j in range(k+1, n):
    Q[j][k] = R[j][k]
    for i in range(k, n):
        Q[i][j] -= Q[i][k] * Q[k][j]

for i in range(n):
    R[i][k] = 0
    for j in range(k, n):
        R[i][j] -= Q[i][k] * R[k][j]

return Q, R

def energy(G):
    A = G['adj_matrix']
    eigenvals = eigenvalues(A)
    return sum(abs(l) for l in eigenvals)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    G = {
        'nodes': list(range(n)),
        'edges': [(random.randint(0, n-1), random.randint(0, n-1)) for _ in range(random.randint(0, n*(n-1)/2))]
    }

    adj_matrix = [[0] * n for _ in range(n)]
    for u, v in G['edges']:
        adj_matrix[u][v] = 1
        adj_matrix[v][u] = 1

    G['adj_matrix'] = adj_matrix

    energy_value = energy(G)

    # Heuristic lower bound on resolution length (simplified example)
    resolution_length = 2 ** math.ceil(energy_value / 2)

    return {
        "metric_name": "Resolution Length",
        "metric_value": resolution_length,
        "instances_tested": 1,
        "conjecture_holds": resolution_length >= 2 ** (energy_value / 2),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i + 1 for i in range(5, 35)]

    results = []
    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r['metric_value'] for r in results) / len(results)
std_value = math.sqrt(sum((r['metric_value'] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

if all(r['conjecture_holds'] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
else:
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
    counterexample = "Resolution length does not grow exponentially with energy"
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00bb6b0c.py", line 112, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00bb6b0c.py", line 93, in run_trial
    energy_value = energy(G)
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00bb6b0c.py", line 75, in energy
    eigenvals = eigenvalues(A)
                ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00bb6b0c.py", line 47, in eigenvalues
    Q, R = qr_decomposition(A)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00bb6b0c.py", line 58, in qr_decomposition
    v[k] += norm_v
    ~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing results | next: Debug qr_decomposition implementation for out-of-range indexing

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	114103
2	propose	ollama_remote	qwen3:8b	0	0	88125
3	propose	ollama_remote	qwen3:8b	0	0	102784
4	propose	ollama_remote	qwen3:8b	0	0	115758
5	propose	ollama_remote	qwen3:8b	0	0	102514
6	preregistration	ollama_remote	qwen3:8b	0	0	24183
7	novelty	ollama_remote	qwen3:8b	0	0	17524
8	novelty	ollama_remote	qwen3:8b	0	0	15152
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10920
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12303
11	judge	ollama_remote	qwen3:8b	0	0	15148

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 618514 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/e90418e8fa5a.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/e90418e8fa5a.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/e90418e8fa5a.tar.gz* (if generated)